

CareerHub — Portal Playbooks

Per-portal field-mapping notes, quirks, and the default engine for company career pages

The autofill engine's confidence-scoring model (previous document) works everywhere, but every major job portal has its own form structure, quirks, and failure points. This document catalogs the priority portals CareerHub should build dedicated field-maps for, plus the fallback "default engine" for the long tail of individual company career pages, which mostly run on a small number of ATS platforms (Workday, Greenhouse, Lever, iCIMS, SmartRecruiters) rather than custom code.

Coverage Tiers

Tier	Portals/Engines	Approach
Tier 1 — Dedicated field-map	Naukri, LinkedIn, Indeed, Foundit, Wellfound	Exact selector maps, near-100% fill accuracy, maintained centrally.
Tier 2 — ATS-pattern map	Workday, Greenhouse, Lever, iCIMS, SmartRecruiters	One map per ATS engine covers thousands of company career pages that run on it.
Tier 3 — Default heuristic engine	Any unrecognized company site	Falls back to the generic scoring engine from the Autofill Spec doc — lower accuracy, always flags for review.

Tier 1 — Named Job Portals

Naukri.com <i>India's largest job portal — highest priority for the Indian jobseeker audience</i>
Coverage priority: Very High — build first Known quirks / issues:

- Multi-step profile wizard (Personal Details -> Education -> Employment -> Resume) rather than one long form — the engine must detect which step is active before filling.
- Resume headline and "Key Skills" use tag-style multi-select inputs, not plain text fields — needs a different fill method (simulate tag-add clicks) than text inputs.
- Education section uses cascading dropdowns (Degree -> Specialization -> University) where each depends on the previous selection loading new options via AJAX.
- Notice period and current/expected salary are dropdown ranges, not free text — must match to nearest bucket, not exact value.

Engine approach:

- Dedicated selector map per wizard step; detect active step via the step-indicator element before choosing which field set to fill.
- For tag inputs, simulate: focus field -> type value -> press Enter/click suggestion -> verify tag appears.
- For cascading dropdowns, fill top-level first, wait for the dependent dropdown to populate (poll for options), then fill it.
- Salary/notice period: round to nearest available bucket and flag for review since exact match is impossible.

LinkedIn (Easy Apply + Full Profile)

High application volume; two very different flows to support

Coverage priority: Very High — build first

Known quirks / issues:

- "Easy Apply" modal is a short multi-step popup (contact info -> resume upload -> screening questions) that unloads and reloads between steps.
- Screening questions are often custom free-text or numeric ("Years of experience with X?") that can't be pre-mapped generically — need a lightweight AI-assisted best-guess from Profile data.
- Full profile edit forms (outside Easy Apply) are heavily component-based (React) with fields that only render once a section is expanded/clicked.
- LinkedIn actively changes DOM structure and class names frequently -- highest maintenance burden of any portal.

Engine approach:

- Treat each Easy Apply step as a separate mini-fill operation; re-scan DOM after every step transition rather than assuming persistence.
- For custom screening questions, use Profile + resume text to generate a suggested answer, but always show it for one-tap confirm rather than auto-submitting free text blindly.
- Use resilient selectors (aria-label, role attributes) over class names, since LinkedIn's class names are obfuscated/rotate.
- Budget for higher-frequency map maintenance and a fast community-reported-breakage pipeline specifically for this portal.

Indeed

Large global reach; simpler structure than LinkedIn but resume-parsing quirks

Coverage priority: High

Known quirks / issues:

- Indeed's own resume parser sometimes pre-fills fields from an uploaded resume with parsing errors (wrong company/date splits) that then sit in the form looking "already filled."
- Some employers use Indeed's native apply form; others redirect to their own ATS (Workday/Greenhouse) mid-flow -- the engine must detect this handoff.
- Phone/location fields sometimes auto-populate from browser/account data, causing double-fill conflicts with the extension.

Engine approach:

- Always run the verification pass even on "already filled" fields -- Indeed's own autofill is not trustworthy and should be treated as unverified, not skipped.
- Detect redirect to a third-party ATS domain mid-application and hand off to that ATS's Tier 2 map instead of continuing with the Indeed map.
- Check for pre-existing values before filling; only overwrite if confidence in Profile data is higher than treating existing value as correct (flag conflicts instead of silently overwriting).

Foundit (formerly Monster India)

Established Indian portal, especially for mid-experience roles

Coverage priority: Medium-High

Known quirks / issues:

- Legacy form architecture (older server-rendered pages mixed with newer React widgets) -- inconsistent field structure across different sections of the same profile.
- Resume upload and "Resume Headline" are separate flows that don't sync automatically.
- Location field uses a searchable dropdown requiring partial-text-then-select rather than direct typing.

Engine approach:

- Maintain two selector sub-maps (legacy vs. new widget sections) rather than assuming one consistent pattern site-wide.
- Treat resume upload and headline as two separate fill actions in the flow, not one.
- For searchable dropdowns: type partial text, wait for the option list to render, click the best text match rather than setting value directly.

Wellfound (formerly AngelList Talent)

Startup-focused -- smaller volume but relevant for tech-savvy jobseekers

Coverage priority: Medium

Known quirks / issues:

- Profile-based apply (no per-application form) -- most "filling" is actually one-time profile setup rather than repeated form fills.
- Heavy use of rich-text bio/pitch fields (contenteditable divs) instead of plain textareas -- standard input-event dispatch doesn't always work on these.
- Salary/equity expectation fields use sliders in some flows -- not fillable via standard value-setting at all.

Engine approach:

- Position this portal primarily under "one-time profile sync" rather than the repeated fill flow -- lower engineering priority for the fill engine itself.
- For contenteditable fields, use the Selection/Range API to insert text rather than a value assignment.
- Sliders: skip auto-fill entirely, flag as "manual input needed" -- not worth the reliability risk of simulating drag events.

Tier 2 & 3 — Default Engine for Direct Company Career Pages

Most "apply on the company's own website" pages are not actually custom-built -- they run on one of a small number of enterprise ATS platforms embedded into the company's career site. Recognizing which ATS is running (from the page URL, embedded script, or DOM fingerprint) lets CareerHub apply one well-tested map to thousands of different companies at once, instead of treating every company site as unknown.

ATS detection method:

- Check the URL pattern first -- e.g. *myworkdayjobs.com*, *boards.greenhouse.io*, *jobs.lever.co*, *icims.com*, *smartrecruiters.com* subdomains are strong signals even before the page fully loads.
- If the company uses a custom domain with an embedded ATS (common for larger companies), check for ATS-specific script tags, iframe sources, or distinctive CSS class prefixes in the page source.
- Fall back to the generic heuristic engine only if no known ATS fingerprint is detected.

ATS	Detection signal	Typical quirks	Engine approach
Workday	URL contains myworkdayjobs.com; distinctive multi-page wizard with a persistent left-side progress tracker.	Long, rigid multi-section wizard (My Information -> My Experience -> Application Questions -> Voluntary Disclosures); heavy use of custom dropdown/combobox widgets that aren't native elements; session timeouts on long forms are common.	Build one robust Workday map: custom combobox widgets need click-to-open then click-option (not direct value set); auto-save progress notification lets the engine safely pause/resume across sections; treat Voluntary Disclosures (gender, veteran status, ethnicity) as opt-in only, never pre-filled without explicit user consent.
Greenhouse	URL contains boards.greenhouse.io or job-boards.greenhouse.io.	Single-page form, generally clean semantic HTML with real elements -- one of the easier ATS platforms to map; custom questions per job posting vary widely.	High baseline heuristic accuracy even without a perfect map since markup is clean; still worth a light dedicated map for the standard fields (name, email, phone, resume upload, LinkedIn URL) which are nearly identical across all Greenhouse-powered postings.
Lever	URL contains jobs.lever.co.	Clean single-page form similar to Greenhouse; resume upload can auto-parse and populate some fields, similar caution as Indeed's parser.	Reuse the same 'don't trust pre-filled values without verification' rule; standard field map for core fields, generic engine for posting-specific custom questions.
iCIMS	URL contains icims.com, or company domain with icims-branded iframe.	Older, more rigid architecture; multi-page with strict validation (exact date formats, mandatory field ordering); session can expire faster than other platforms.	Prioritize strict format compliance (dates, phone patterns) since iCIMS validation is stricter than most -- a malformed autofill is more likely to visibly error here than elsewhere, which is actually helpful for catching mistakes early.

ATS	Detection signal	Typical quirks	Engine approach
SmartRecruiters	URL contains jobs.smartrecruiters.com.	Modern single-page form, similar in cleanliness to Greenhouse/Lever; increasingly uses AI-driven conversational-style question flows for screening.	Standard field map for core fields; conversational/AI screening questions fall back to the same 'suggest, don't auto-submit' approach used for LinkedIn's custom questions.

Default fallback engine (true unknowns):

If no portal- or ATS-specific map matches, CareerHub always has a working fallback: the generic confidence-scoring engine described in the Autofill Engine Spec (label/name/autocomplete matching with the synonym dictionary). This guarantees the extension never completely fails on an unrecognized site -- it simply operates at lower confidence and flags more fields for manual review, which is the correct, safe degradation rather than doing nothing or guessing recklessly.

Field-Map Maintenance Workflow

Since portals change their HTML periodically, field-maps need a lightweight update loop that doesn't require republishing the extension:

- 1. Central field-map store (server-side JSON per domain/ATS, versioned)
- 2. Extension fetches latest maps on startup + periodic refresh (not bundled at build time)
- 3. User hits a broken field -> one-tap "Report issue" -> sends {domain, fieldMapVersion, selector} (never sends the user's actual personal data in the report)
- 4. Reports aggregate on the backend -> flags maps with rising failure rates for manual review
- 5. Maintainer patches the JSON map -> pushes new version -> all users get it on next refresh, no extension store update/review cycle needed

Build priority recommendation:

Order	Target	Why
1	Naukri + LinkedIn Easy Apply	Highest usage among Indian jobseekers/students -- validates the core value prop fastest.
2	Workday + Greenhouse (ATS-level)	Covers the largest share of direct company career pages in one build each.
3	Indeed, Lever, SmartRecruiters	Broaden coverage once the core flow is proven.
4	Foundit, iCIMS, Wellfound	Lower volume but rounds out coverage for specific user segments.

This tiered approach means CareerHub never has to choose between "deep support for a few portals" and "broad support everywhere" -- the ATS-level maps make broad coverage nearly free once the top few platforms are mapped, since one Workday map instantly works across every company that happens to use Workday.